

PHƯƠNG PHÁP CHIA ĐỂ TRỊ (DIVIDE AND CONQUER)

PGS.TS. Đặng Thị Oanh

Ngày 1 tháng 3 năm 2026

Tài liệu tham khảo

Chapter 5

Anany Levitin, **Introduction to the Design and Analysis of Algorithms**, ebook.

Review the Previous Lesson

- 1 Tổng quan về đệ quy
- 2 Lựa chọn giải thuật đệ quy
- 3 Tính đúng đắn của giải thuật đệ quy
- 4 Đánh giá độ phức tạp của giải thuật đệ quy
- 5 Khử đệ quy

Nội dung

- 1 Chiến lược chia để trị
- 2 Mô hình, lược đồ chia để trị
- 3 Ưu, nhược điểm của chia để trị
- 4 Một số Bài toán giải bằng chia để trị
- 5 Bài tập

Chiến lược chia để trị

Chiến lược chia để trị là kỹ thuật thiết kế thuật toán dựa trên đệ quy.

Chiến lược chia để trị

Chiến lược chia để trị là kỹ thuật thiết kế thuật toán dựa trên đệ quy.

Là phương pháp giải quyết bài toán phức tạp bằng cách chia nhỏ thành các bài toán con đồng dạng, dễ xử lý hơn, sau đó giải quyết từng bài toán con và kết hợp kết quả.

Chiến lược chia để trị

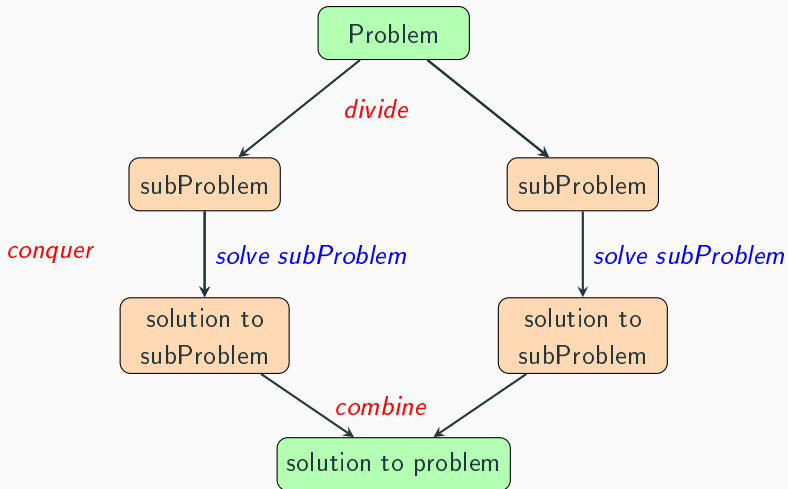
Chiến lược chia để trị là kỹ thuật thiết kế thuật toán dựa trên đệ quy.

Là phương pháp giải quyết bài toán phức tạp bằng cách chia nhỏ thành các bài toán con đồng dạng, dễ xử lý hơn, sau đó giải quyết từng bài toán con và kết hợp kết quả.

Quy trình 3 Bước

- ➊ **Chia (Divide):** Chia bài toán lớn thành các bài toán con đồng dạng.
- ➋ **Trị (Conquer):** Giải quyết các bài toán con. Nếu chúng đủ nhỏ, giải trực tiếp; trái lại, tiếp tục chia nhỏ.
- ➌ **Kết hợp (Combine):** Tổng hợp các kết quả của bài toán con → lời giải bài toán ban đầu.

Mô hình chia để trị



Lược đồ giải thuật chia để trị

- Consider $DC(R)$ is algorithm of problem P with input range R .
- If R can be partitioned into n sub-ranges ($R = R_1 \cup R_2 \cup \dots \cup R_n$) and $DC(R_0)$ has solution

- **Diagram:**

$DC(R) \equiv$

if ($R = R_0$)

Get solution $DC(R_0)$;

else

Partition R into $R_1, R_2, \dots, R_n$

for ($i = 1$ to n)

$DC(R_i)$;

Combine the solutions.

End.

Tính đúng đắn của thuật toán DC được chứng minh bằng quy nạp.

Ưu/nhược điểm và khắc phục

- **Ưu điểm:** Giải quyết hiệu quả các bài toán phức tạp, tối ưu thời gian chạy.
- **Nhược điểm**
 - Tốn tài nguyên bộ nhớ và nguy cơ tràn Stack;
 - Hiệu suất kém khi có bài toán con trùng lặp;
 - Cài đặt và quản lý phức tạp.
- **khắc phục:** Khử đệ quy (iterative) hoặc lưu trữ kết quả bài toán con (memoization).

Ứng dụng của chiến lược chia để trị

- Sorting: Mergesort and Quicksort
- Tree traversals
- Binary search
- Matrix multiplication - Strassen's algorithm
- Closest Pair of Points
- Convex hull - QuickHull algorithm
- Delaunay Triangulation, Voronoi Diagram

Ứng dụng của chiến lược chia để trị

- Sorting: Mergesort and Quicksort
- Tree traversals
- Binary search
- Matrix multiplication - Strassen's algorithm
- Closest Pair of Points
- Convex hull - QuickHull algorithm
- Delaunay Triangulation, Voronoi Diagram

**Chiến lược CHIA ĐỂ TRỊ không chỉ áp dụng
trong thiết kế thuật toán mà còn là
TƯ DUY QUẢN LÝ!**

Một số ví dụ

- Binary Search
- Mergesort

Binary Search

- **Input:** Mảng $a(1, 2, \dots, n)$, giá trị x ; (a đã sắp xếp).
- **Output:** Vị trí của x trong mảng a .

Các bước thực hiện

- 1 **Chia:** Tìm phần tử giữa (mid) của mảng hiện tại.
- 2 **Trị:**
 - Nếu $key = arr[mid]$: Tìm thấy và kết thúc.
 - Nếu $key < arr[mid]$: Tìm kiếm tiếp ở nửa bên trái (0 đến $mid - 1$).
 - Nếu $key > arr[mid]$: Tìm kiếm tiếp ở nửa bên phải ($mid + 1$ đến cuối)
- 3 **Kết hợp:** Không cần kết hợp trong tìm kiếm nhị phân, vì kết quả trả về ngay khi tìm thấy.

Chỉ một nửa mảng con được xử lý tiếp, nên đây thuộc loại "giảm để trị".

Binary Search

Binary search diagram (Recursive Pseudocode)

$BinarySearch(a, x, L, R): \text{int} \equiv$

if $L > R$: return -1. // Target not found

$mid = L + (R - L)/2$;

if $a[mid] == x$: return mid

else if $x < a[mid]$:

return $BinarySearch(a, x, L, mid - 1)$

else: return $BinarySearch(a, x, mid + 1, R)$.

- **Độ phức tạp thời gian:** $O(\log n)$.

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ T(n/2) + 1 & \text{if } n > 1 \end{cases}$$

$\rightarrow T(n) = O(\log n)$.

- **Độ phức tạp không gian:** $O(\log n)$. Đệ quy tạo ra stack frames cho mỗi lần gọi. Độ sâu của đệ quy tỉ lệ thuận với $\log n$.

Binary Search

Binary search diagram (Iterative Pseudocode)

BinarySearch(a, x, L, R): int $\equiv$

while $L \leq R$ **do**

$mid := L + \text{floor}((R - L)/2)$

if $a[mid] < x$ **then**

$L := mid + 1$

else if $a[mid] > x$ **then**

$R := mid - 1$

else:

return mid // Element found

return unsuccessful // Element not found

- **Độ phức tạp thời gian:** $O(\log n)$. Vì ở mỗi bước, phạm vi tìm kiếm được chia đôi, số lần so sánh tối đa là $\log_2 n$.
- **Độ phức tạp không gian:** $O(1)$. Phương pháp lặp sử dụng một lượng bộ nhớ không đổi (chỉ lưu chỉ số L, R, mid) bất kể kích thước mảng đầu vào, tối ưu hơn phương pháp đệ quy.

Binary Search

Recursive and Iterative strategy for Binary search

- **Recursive strategy**

- Ưu: Mã nguồn dễ hiểu, ngắn gọn và tường minh.
- Nhược: Tốn bộ nhớ hơn so với lặp do lưu trữ các lời gọi hàm trong ngăn xếp.

- **Iterative strategy**

- Ưu điểm: Cực kỳ hiệu quả cho các danh sách lớn đã được sắp xếp.

Merge Sort

- **Divide:** Chia danh sách chưa được sắp xếp gồm n phần tử thành n danh sách con, mỗi danh sách bao gồm 1 phần tử.

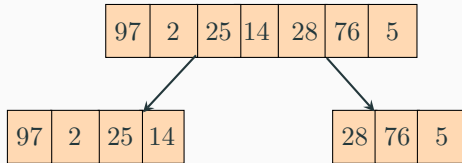
Conquer: Một danh sách chỉ có một phần tử được coi là đã được sắp xếp.

Merge: Liên tục hợp nhất các danh sách con để tạo thành danh sách con mới đã được sắp xếp cho đến khi chỉ còn một danh sách con duy nhất. Đây là danh sách đã được sắp xếp.

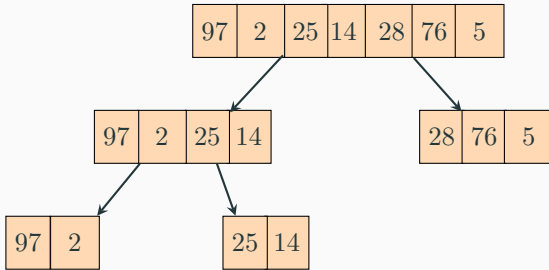
Mô phỏng Merge Sort by recursion

97	2	25	14	28	76	5
----	---	----	----	----	----	---

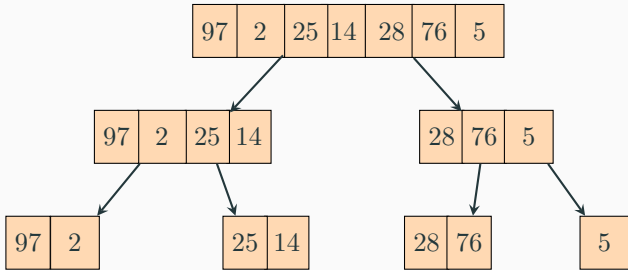
Mô phỏng Merge Sort by recursion



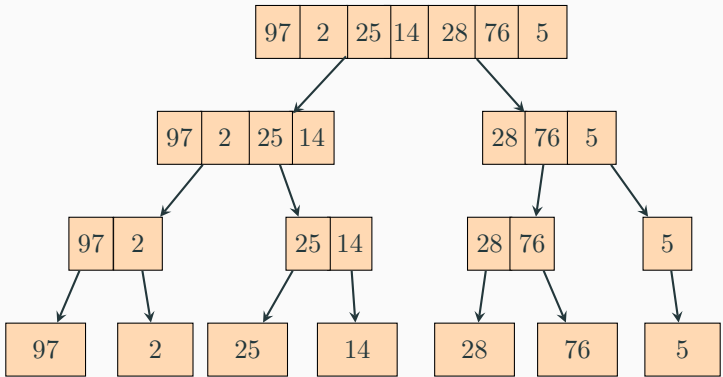
Mô phỏng Merge Sort by recursion



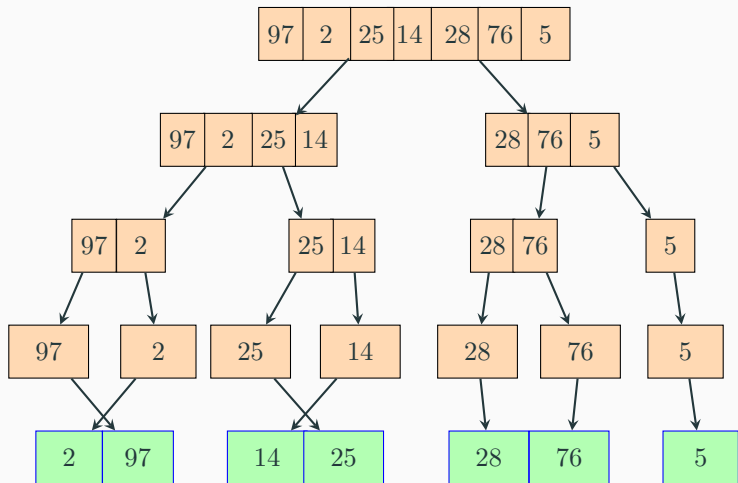
Mô phỏng Merge Sort by recursion



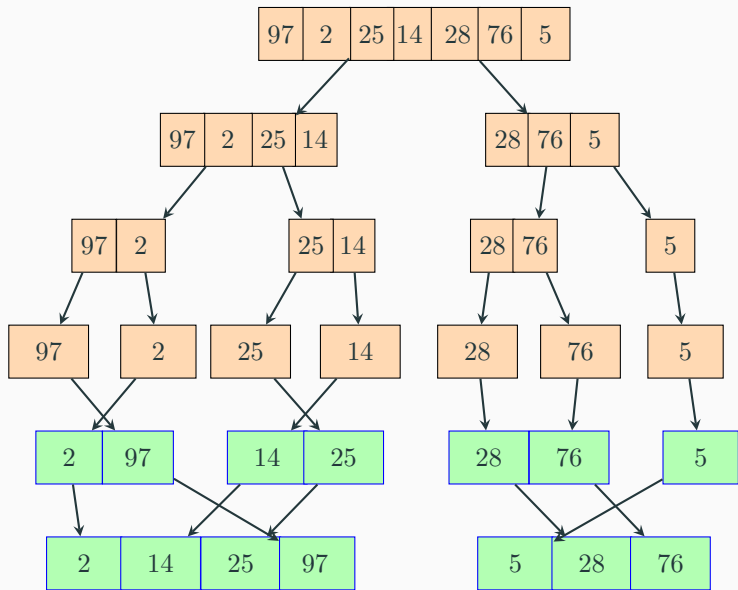
Mô phỏng Merge Sort by recursion



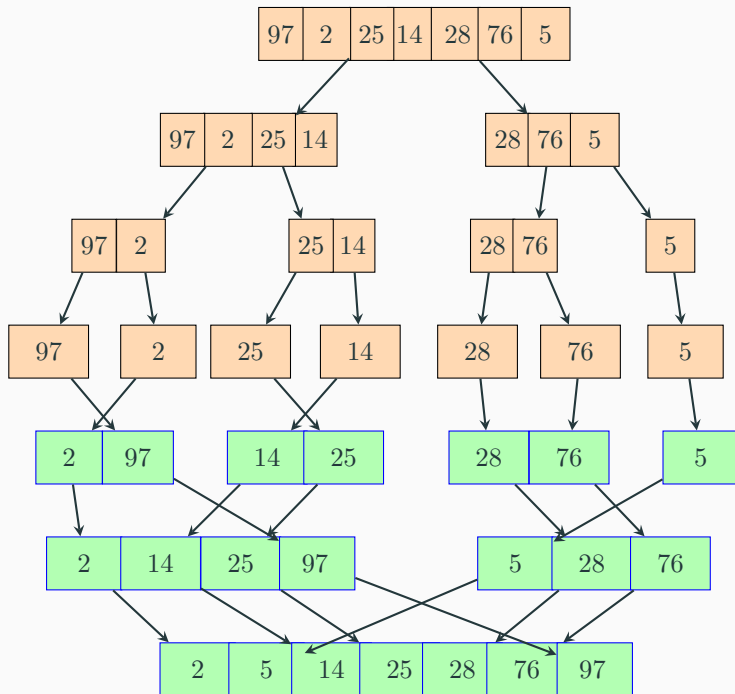
Mô phỏng Merge Sort by recursion



Mô phỏng Merge Sort by recursion



Mô phỏng Merge Sort by recursion



Bài toán MinMax

Bài toán: Tìm Max và Min của một dãy A . Bao nhiêu phép so sánh?

- **Input:** dãy $A(1, 2, \dots, n)$.
- **Output:** Giá trị Min và Max của mảng A .
- **Ý tưởng:** Chia mảng thành hai nửa đệ quy, tìm max/min của từng nửa, sau đó so sánh kết quả của hai nửa để tìm ra kết quả cuối cùng.

MinMax - Chiến lược DC

Các bước thực hiện

- **Chia:** Chia mảng $A[l, \dots, r]$ thành hai phần: $A[l, \dots, mid]$ và $A[mid + 1, \dots, r]$ với $mid = (l + r)/2$.
- **Trị:**
 - 1 T/h cơ sở 1 (1 phần tử): $l = r$. $Max = Min = A[l]$.
 - 2 T/h cơ sở 2 (2 phần tử): $l + 1 = r$. So sánh $A[l]$ và $A[r]$ để tìm Max và Min.
 - 3 Trường hợp tổng quát ($n > 2$): Gọi đệ quy cho hai nửa để tìm (max_1, min_1) và (max_2, min_2) .
- **Kết hợp:**
 - 1 Max tổng thể $= \max(max_1, max_2)$.
 - 2 Min tổng thể $= \min(min_1, min_2)$.

MinMax - DC

Algorithm diagram

```
MinMax(a,L,R):(int,int)  $\equiv$   
  if ( $R - L \leq 1$ )  
    return(min(aL,aR),max(aL,aR));  
  else  
    ( $\min_1, \max_1$ ) =  $MinMax(a, L, (L + R)/2)$ ;  
    ( $\min_2, \max_2$ ) =  $MinMax(a, (L + R)/2 + 1, R)$ ;  
    return(min( $\min_1, \min_2$ ), max( $\max_1, \max_2$ ));  
  endif;  
End.
```

The first call with list has n elements: $MinMax(a,0,n-1)$.

MinMax - DC

Prove the correctness: Induction by size of the list $n = R - L + 1$.

- Base case: $n = R - L + 1 \leq 2$ (list has 1 or 2 elements)
- Inductive assumption: Algorithm correct with all list has $n = R - L + 1$ ($n > 2$).
 - Mean $\text{MinMax}(a, L, R)$ return the correct min, max values in interval $[L..R]$ (from a_L to a_R)
- General case: Prove the algorithm correct with $n+1 = R - L + 2$
 - Apply induction assumption to each recursive call.
 - $\text{MinMax}(a, L, (L+R)/2)$;
 - $\text{MinMax}(a, (L+R)/2+1, R)$;

* Time running complexity

$$T(n) = \begin{cases} 2 & \text{if } n \leq 2 \\ 2T(n/2) + 2 & \text{if } n > 2 \end{cases}$$

matrix multiplication

Bài toán

- Input: Two matrices A, B size $n \times n$
- Output: C size $n \times n$, $C = A \times B$

DC Methodology: The matrices A and B are partitioned into four $n/2 \times n/2$ submatrices

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$$

$$B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

matrix multiplication

Recursive Step: The product is calculated by computing eight sub-problems:

- $C_{11} = A_{11}B_{11} + A_{12}B_{21}$
- $C_{12} = A_{11}B_{12} + A_{12}B_{22}$
- $C_{21} = A_{21}B_{11} + A_{22}B_{21}$
- $C_{22} = A_{21}B_{12} + A_{22}B_{22}$.

Base Case: The recursion continues until the matrices are small (e.g. 1×1 or 2×2), at which point they are multiplied directly.

Complexity: Standard divide-and-conquer yields a recurrence relation of $T(n) = 8T(n/2) + O(n^2)$, resulting in $O(n^3)$ time complexity, which is the same as the naive approach.

matrix multiplication

Bài toán

- Input: Two matrices A , B size $n \times n$
- Output: C size $n \times n$, $C = A \times B$

DC Methodology: The matrices A and B are partitioned into four $n/2 \times n/2$ submatrices

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$$

$$B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

matrix multiplication

Recursive Step: The product is calculated by computing eight sub-problems:

- $C_{11} = A_{11}B_{11} + A_{12}B_{21}$
- $C_{12} = A_{11}B_{12} + A_{12}B_{22}$
- $C_{21} = A_{21}B_{11} + A_{22}B_{21}$
- $C_{22} = A_{21}B_{12} + A_{22}B_{22}$.

Base Case: The recursion continues until the matrices are small (e.g. 1×1 or 2×2), at which point they are multiplied directly.

Complexity: Standard divide-and-conquer yields a recurrence relation of $T(n) = 8T(n/2) + O(n^2)$, resulting in $O(n^3)$ time complexity, which is the same as the naive approach.

matrix multiplication

Strassen's Algorithm Improvement

Conquer: Seven specific products (M_1 to M_7) of these submatrices are recursively computed. This is where Strassen's trick lies, reducing the number of multiplications from eight to seven.

- $M_1 = (A_{11} + A_{22}) * (B_{11} + B_{22})$

- $M_2 = (A_{21} + A_{22}) * B_{11}$

- $M_3 = A_{11} * (B_{12} - B_{22})$

- $M_4 = A_{22} * (B_{21} - B_{11})$

- $M_5 = (A_{11} + A_{12}) * B_{22}$

- $M_6 = (A_{21} - A_{11}) * (B_{11} + B_{12})$

- $M_7 = (A_{12} - A_{22}) * (B_{21} + B_{22})$

matrix multiplication

Strassen's Algorithm Improvement (continue)

Combine: The four submatrices of the resulting matrix C are computed by adding and subtracting the seven products (M_1 to M_7), and then concatenated to form the final $n \times n$ result matrix.

- $C_{11} = M_1 + M_4 - M_5 + M_7$
- $C_{12} = M_3 + M_5$
- $C_{21} = M_2 + M_4$
- $C_{22} = M_1 - M_2 + M_3 + M_6$

BÀI TẬP